

AI PROJECT MANAGER

Universal Workflow Document

Valid for: Claude, Gemini, Codex, or any AI assistant

PRIME DIRECTIVE — READ BEFORE ANYTHING ELSE

Any AI reading this document must follow this workflow exactly, in order, every session. Do not write a single line of code, create any file, or make any design decision until all Phase 0 questions have been asked and answered. This document is the law of the project. If a HANDOFF.md exists in this project, read it completely before asking any questions.

HOW TO USE THIS DOCUMENT

For the human:

- Start every new project by opening a session and saying: *"Read AI_PROJECT_MANAGER.md and start a new project"*
- Start every existing project session by saying: *"Read AI_PROJECT_MANAGER.md and HANDOFF.md and let's continue"*
- Never skip phases — they exist to prevent going back to fix small things that break big things

For the AI:

- Read this entire document before responding
- If a HANDOFF.md exists, read it and confirm you understand the current state
- If an AI context file exists for your platform (e.g. `CLAUDE.md`, `GEMINI.md`, `CODEX.md`), read it
- Ask ALL Phase 0 questions before doing anything
- Generate the AI context file for your platform at the end of Phase 2 setup (see Phase 2)

- Generate or update HANDOFF.md at the end of every session
 - One module per session where possible — do not mix concerns
 - Never edit a file you have not been shown in full first
-
-

PHASE 0 — SESSION OPENING PROTOCOL

(AI Must Complete This Every Single Session)

STEP 0A — AI Reads Context First

Before asking anything, the AI must:

- Confirm this document has been read in full
 - Check if `HANDOFF.md` exists and read it completely
 - Check if an AI context file exists for this platform:
 - Claude → `CLAUDE.md`
 - Gemini → `GEMINI.md`
 - Codex → `CODEX.md`
 - Other → `[AINAME].md`
 - Note the current project phase from `HANDOFF.md`
 - Note what was completed last session
 - Note what is blocked or pending
-

STEP 0B — AI Asks These Questions (In This Order)

The AI must ask the following before writing any code or making any decisions. Ask them conversationally — do not dump them all at once.

Group 1 — Project Identity (New Projects Only)

1. What is the project name?
2. What type of project is this? (Web app / PC app / Python script / Android / iOS / other)
3. Who is the target user? Describe them in one sentence.
4. What problem does this solve? Describe it in one sentence.
5. What does “done” look like? What are the hard scope boundaries?
6. What platforms and OS versions must this support?

Group 2 — Session Goals (Every Session)

1. What is the goal for THIS session specifically?
2. Which module or file are we working on today?
3. Are there any blockers or issues from last session?
4. Has anything changed in the project since the last session?
5. Are we in design phase, build phase, or polish phase right now?

Group 3 — Pre-Build Brainstorm (New Projects or New Features Only)

1. Let’s list every core component, screen, and feature — confirm nothing is missing before building
 2. Are there any third-party libraries or APIs this will depend on?
 3. Is there a data model or database involved — has the schema been designed?
 4. What talks to what — map out the integration points
 5. Are there any security concerns we need to address upfront?
-



STEP 0C — AI Confirms Before Proceeding

After questions are answered, AI must state:

- Summary of what it understands about the project
- What it is going to do THIS session
- Which files it will touch
- What it will NOT touch this session

- Then wait for confirmation before writing anything
-
-



PHASE 1 — PROJECT IDENTITY

(Lock Everything Here Before Any Design or Code)

- **Project / App name finalized** — no changing this later
- **Tagline or one-sentence description written**
- **Logo designed and exported** (all required sizes and formats)
- **App icon designed:**
 - Windows: `.ico` (16, 32, 48, 256px)
 - Android: mdpi through xxxhdpi + adaptive icon
 - iOS: all required sizes
 - Web: favicon (multi-size `.ico` or SVG)
- **Color scheme locked:**
 - Primary color: _____
 - Secondary color: _____
 - Accent color: _____
 - Background color: _____
 - Surface / card color: _____
 - Error color: _____
 - Warning color: _____
 - Success color: _____
 - Dark mode variants defined (or explicitly out of scope — document that)
- **Font hierarchy locked:**

- Display / heading font: _____
 - Body font: _____
 - Monospace font (if applicable): _____
 - H1 size: ____ H2: ____ H3: ____ Body: ____ Caption: ____ Label: ____
 - **Icon set chosen** — consistent style throughout entire app
 - **Brand voice / tone defined** (professional / casual / technical / friendly)
 - **Naming conventions written down:**
 - Files: _____
 - Functions: _____
 - Variables: _____
 - Git branches: _____
 - All identity assets exported and stored in `/assets` before coding starts
 - **Identity signed off — do not change after this point**
-
-

PHASE 2 — PROJECT SETUP

(Before First Commit — Do This Once, Do It Right)

Repository & File Hygiene

- Repository created
- `.gitignore` configured — no secrets, no venv, no build artifacts, no `.env` files
- `.env.example` committed — shows all required variables, no actual values
- `README.md` created — what it is, how to run it, how to build it
- `CHANGELOG.md` started — even a placeholder entry
- `AI_PROJECT_MANAGER.md` copied into project root

- `HANDOFF.md` created from template (see HANDOFF template section at bottom)
- **AI context file generated by the AI for this platform** (see format below)
- Version number defined in ONE place — referenced everywhere else from there

AI Context File (Generated by AI at End of Phase 2)

The AI must generate a context file named for its own platform at the end of Phase 2. This file is project-specific and lives in the project root.

Filename:

- Claude → `CLAUDE.md`
- Gemini → `GEMINI.md`
- Codex → `CODEX.md`
- Other → `[AINAME].md`

Contents the AI must include:

```
# [AI NAME] - [Project Name] Context File
**Generated:** [Date]
**Project type:** [Web / Desktop / Mobile / Script]
**AI platform:** [Claude / Gemini / Codex / etc.]

## Project Summary
[One paragraph - what this project is, who it's for, what it does]

## Project Structure
[File/folder tree of the project as it currently stands]

## Current Phase
[Phase number and name]

## Key Rules for This Project
- Always read HANDOFF.md before starting any session
- Always read AI_PROJECT_MANAGER.md if starting fresh
- One module per session - do not mix concerns
- Never edit a file without seeing it in full first
- Update HANDOFF.md at the end of every session

## Modules Overview
[Brief one-line description of each module as they are built]
```

```
## Important Decisions Made
[Summary of key architectural decisions — pulled from HANDOFF.md decision log]

## Do Not Touch Without Explicit Instruction
[Any files or modules that are locked / complete / fragile]

## Environment Notes
[Ports, env vars, paths, anything needed to run the project]
```

This file is updated by the AI whenever a significant change is made to project structure or direction.

Environment

- Dev / staging / production configs separated from day one
- Dependencies pinned (`requirements.txt` , `package-lock.json` , etc.)
- Virtual environment or container defined — not “just install it globally”
- Secrets and API keys in environment variables — never hardcoded, ever
- Port numbers documented — no conflicts with other running services

Version Control Workflow

- Branching strategy decided:
 - Simple: `main + dev + feature/name`
 - Or: keep it simple with just `main` and feature branches
- Commit message format defined — descriptive, no `fix stuff` or `wip`
- Tag releases with version numbers (`v1.0.0` , `v1.0.1` , etc.)
- Never commit directly to `main` on anything going to production
- Merge = tested and verified, not just “it ran once”

Logging & Error Handling (Set Up Day One)

- Logging strategy decided — log levels: `DEBUG` / `INFO` / `WARN` / `ERROR`
- Log file location defined and documented

- Log rotation configured — logs don't grow forever
- Unhandled exceptions caught at top level — logged, never silently swallowed
- User-facing errors show helpful message — never a raw stack trace
- Build order documented — if modules must be built in sequence, write it down

Backup & Sync Strategy

- Single source of truth defined — GitHub / network drive / OneDrive, pick one
 - Build artifacts location documented
 - Sync strategy across machines documented (especially for multi-machine workflow)
-

PHASE 3 — MODULAR ARCHITECTURE PLAN

(Design the Structure Before Writing the Structure)

Core Rules

- UI layer completely separated from business logic — they do not mix
- Data / API layer is its own isolated module — swappable without touching UI
- Config and constants isolated in one place — no magic numbers anywhere in code
- Each module has ONE clear job — if you can't describe it in one sentence, split it
- Folder structure organized by feature/module — not by file type
- Shared utilities in their own module — never copy-pasted across files
- New features added as new modules — never hacked into existing ones

Module Map (Fill This Out Before Building)

Document every planned module:


```

project_name/
├─ main.py           # Entry point only – no logic here
├─ app.py            # Main window / shell – layout and glue only
├─ constants.py      # All constants and config values
├─ config.py         # Settings load/save
├─
├─ ui/
│   ├─ __init__.py
│   ├─ [module].py    # Each screen or panel = its own file
│   └─ dialogs.py     # All dialogs and popups
├─
├─ core/
│   ├─ __init__.py
│   └─ [feature].py   # Each feature = its own file
├─
├─ data/
│   ├─ __init__.py
│   └─ [model].py     # Data models and DB access
├─
└─ utils/
    ├─ __init__.py
    └─ [utility].py   # Shared helpers

```

Code Hygiene Rules

- No circular imports or dependencies between modules
- Interfaces between modules defined — what goes in, what comes out
- Dead code removed — no permanent commented-out blocks
- No hardcoded file paths — built dynamically or pulled from config
- Every function and module named clearly — no mystery abbreviations
- Variable names describe what they hold — not `x`, `temp`, `data`

Dependency Map

Document which modules depend on which — so you know what breaks if something changes:

```

Module A → depends on → Module B, Module C
Module B → depends on → Module D
Module C → depends on → nothing (standalone)

```

AI-Assisted Editing Rules

- Each module is self-contained enough to hand to an AI in isolation
 - `HANDOFF.md` always current before any AI session
 - One module per AI session where possible — no mixing concerns
 - After any AI edit, verify module boundary was not broken
 - Never let AI edit a file it has not been shown in full first
 - AI updates `HANDOFF.md` at end of every session — no exceptions
-
-

PHASE 4 — DESIGN

(All Screens and States Locked Before Any Code Is Written)

Identity Applied to Design

- Color scheme from Phase 1 applied to all mockups
- Font hierarchy from Phase 1 applied consistently
- Icon set from Phase 1 used throughout
- Logo and app icon in place in mockups

Screen Inventory

List every screen/view that exists in the app — every single one:

- _____
- _____
- _____

Every Screen Must Have These States Designed

For each screen listed above:

- Default / normal state
- Loading state (spinner, skeleton, progress bar)
- Empty state (no data, first launch, no results)
- Error state (inline error, full-page error, network failure)
- Success / confirmation state
- Disabled state for all interactive elements

UI Standards

- Consistent spacing/padding scale (4 / 8 / 16 / 24 / 32px)
- Hover states on all interactive elements
- Focus / active states on all buttons and inputs
- Disabled states on all buttons and inputs
- Cursor changes correct — pointer on buttons, grab on draggable, resize on edges
- Tooltip behavior defined
- Modal / dialog behavior defined:
 - Close on backdrop click?
 - Close on ESC key?
 - X button always present?
- Text overflow handled — truncation vs wrap — intentional per element
- Long strings don't break layout (usernames, file paths, error messages)
- Z-index / layering audited — modals always on top, tooltips not clipped
- Scrollbar styling considered
- Animations and transitions intentional — not browser defaults
- Accessibility contrast ratios checked

- Accessibility font size minimums met
- UI scales correctly at non-default zoom / DPI levels
- Print stylesheet considered

Design Sign-Off

- Every screen mocked and reviewed
 - All states designed
 - **Design is frozen — no changes after this point without a documented reason**
 - Build begins only after this checkbox is checked
-
-



PHASE 5 — SECURITY REVIEW

(Before Any Code That Touches Data, Network, or Files)

Input & Data

- All user input sanitized — nothing goes raw into a database, file system, or command
- SQL injection prevention (parameterized queries — never string concatenation)
- File upload validation — type, size, extension all checked server-side
- Sensitive data identified — what is it, where does it live, how is it protected
- Sensitive data not stored in plain text — encrypted at rest
- Sensitive data not logged — passwords, tokens, keys never appear in logs
- Sensitive data not exposed in URL parameters

Authentication & Authorization (If Applicable)

- Authentication strategy decided upfront — not bolted on later
- Password hashing — bcrypt or Argon2, never MD5 or SHA1

- Session tokens — secure, httpOnly, sameSite cookies
- Token expiration defined
- Role-based access control planned before build
- Password reset flow secure — time-limited tokens, single use

Network & API

- HTTPS enforced everywhere — no cleartext HTTP
- API keys and secrets in environment variables — never in code or URLs
- CORS configured correctly — not wildcard `*` in production
- Rate limiting on API endpoints
- External links use `rel="noopener noreferrer"`

Dependencies

- All third-party packages reviewed — actively maintained, no known critical CVEs
- Dependency versions pinned
- Vulnerability scanning plan — how will you know when a dep has a new exploit
- Licensing checked — no GPL in a commercial project without understanding implications

Desktop App Specific

- App does not require admin / UAC elevation unless absolutely necessary
- If elevation required — documented why and scoped minimally
- Config files don't store plaintext credentials
- Compiled executable — code signing considered to reduce AV false positives

Mobile Specific

- Sensitive screens use FLAG_SECURE (Android) or equivalent (iOS) — no recent apps screenshot leak
- Keychain / KeyStore used for credential storage — not SharedPreferences or

UserDefaults

- Privacy usage descriptions accurate in app manifests
 - Network security config enforced
-
-

PHASE 6 — BUILD

(Now We Write Code — In Module Order, UI Shell First)

Build Order (Always Follow This Sequence)

1. **Shell first** — build the complete UI with no logic. Every screen, every button, every panel. Looks real, does nothing.
 2. **Core modules** — build each core logic module in isolation, tested standalone
 3. **Wire up** — connect UI to core logic one module at a time
 4. **Integration** — make everything talk together
 5. **Polish** — all the checklist items below
-

Platform-Specific Build Checklists

WEB APP

Mouse / Input

- Right-click context menus — default suppressed if custom used, left as-is if not
- Scroll wheel — page, inner div, zoom, sliders all behave correctly
- Middle-click on links — opens in new tab (never break this)
- Click vs double-click — defined, no conflict

- Drag and drop — tested on touch too
- Text selection — if disabled anywhere, it's intentional and documented

Keyboard

- Ctrl+C / Ctrl+V — not blocked in inputs
- Ctrl+Z / Ctrl+Y — undo/redo works
- Ctrl+A — select all in inputs
- Ctrl+S — save if applicable
- Ctrl+F — browser find not hijacked
- Enter submits forms
- Tab order logical through all fields
- Escape closes modals / dropdowns / dialogs
- Arrow keys work in dropdowns, sliders, menus

Browser Behavior

- Back/forward buttons work (pushState managed)
- Page refresh — no unexpected state loss
- Deep linking / bookmarkable URLs
- Browser zoom (Ctrl+ / Ctrl-) — layout holds
- 404 page designed and working
- Session timeout handled gracefully — not a white screen
- Console errors cleared before ship

Storage & Data

- LocalStorage vs sessionStorage — chosen intentionally
- Sensitive data NOT in localStorage
- Network failure — clear message, no silent failure
- Offline state handled or PWA/service worker if applicable

Forms

- Validation on submit AND inline
- Required fields clearly marked
- Error messages specific — not just “invalid”
- Success feedback shown
- Duplicate submission prevented (button disabled on submit)
- Autocomplete attributes correct
- Password fields have show/hide toggle

Performance

- Images compressed — WebP preferred
- Lazy loading below the fold
- No blocking scripts in `<head>`
- Font loading strategy — no invisible text flash

Misc

- Mobile viewport meta tag set
- Touch targets minimum 44x44px
- Pinch-to-zoom not disabled
- Copy to clipboard shows feedback
- External links — `rel="noopener noreferrer"`



PC APPLICATION (Windows)

Menu Bar (Day One — Always)

- **File** — New, Open, Save, Save As, Export, Recent Files, Exit
- **Edit** — Undo, Redo, Cut, Copy, Paste, Select All, Find, Preferences
- **View** — Zoom In, Zoom Out, Reset Zoom, Dark/Light toggle, Fullscreen

- **Help** — About, Documentation, Check for Updates, Report a Bug

Mouse

- Right-click context menus on all relevant elements
- Scroll wheel — lists, text areas, canvas, sliders
- Double-click to open/select/rename
- Click and drag — panels, items, elements
- Middle-click if applicable

Keyboard

- Ctrl+C, Ctrl+X, Ctrl+V
- Ctrl+Z / Ctrl+Y — undo/redo (depth limit defined)
- Ctrl+S — save
- Ctrl+Shift+S — save as
- Ctrl+N — new
- Ctrl+O — open
- Ctrl+W — close tab/window
- Ctrl+Q / Alt+F4 — quit
- Ctrl+A — select all
- Ctrl+F — find
- F2 — rename
- Delete — delete selected
- Enter — confirm
- Escape — cancel / close
- Tab / Shift+Tab — focus navigation
- Arrow keys in lists, trees, menus

Window

- Minimize, maximize, restore, close all work
- Window remembers last size and position
- Minimum window size set — can't resize below usable
- Multi-monitor handled correctly
- DPI scaling tested: 100%, 125%, 150%, 200%
- Fonts don't go microscopic or giant at non-standard DPI
- Taskbar icon correct
- System tray icon + right-click menu (if applicable)

File Handling

- Open dialog filters correct types
- Save dialog defaults to last used directory
- Drag-and-drop files into app
- Unsaved changes prompt on close
- Recent files list
- File association registered if applicable

App Behavior

- No frozen window on startup
- Single vs multi-instance — decided intentionally
- Config saves on close — not just on explicit save
- First launch without config — clean defaults, no crash
- Config file location documented
- Crash recovery — restore last session
- Clipboard works after app loses and regains focus
- No UAC elevation unless necessary
- Global hotkeys unregistered on close
- UI updates main thread only — background work on worker threads

Install / Uninstall Wizard

Installer:

- Welcome screen — app name and version
- EULA / license screen
- Install directory selection with default
- Optional components if applicable
- Progress bar during install
- Desktop shortcut — opt-in checkbox, not forced
- Start menu entry — opt-in checkbox
- Launch app checkbox on finish
- Silent install `/s` flag

Uninstaller:

- Listed in Add/Remove Programs — correct name, version, icon
 - Removes all installed files
 - Removes all registry entries created
 - Prompts about AppData/config — user chooses keep or delete
 - Removes shortcuts
 - Confirms completion
 - No orphaned folders or files
-

PYTHON / CUSTOMTKINTER

Menu Bar

- Native Tkinter menu — not fake button menus
- File / Edit / View / Help structure

UI Scaling

- `ctk.set_widget_scaling()` called correctly
- `ctk.set_window_scaling()` called correctly
- Tested at 100%, 125%, 150%, 200% Windows display scaling
- No clipping, overlap, or overflow at any scale
- Fonts scale correctly

Threading

- All UI updates on main thread only
- Background tasks on worker threads
- `after()` or queue for thread-safe UI updates
- No frozen app on long operations — progress shown

Window

- Minimum size set
- Size/position remembered on reopen
- Window icon set (`.ico`)
- Version in title bar or About dialog

File / Config

- Config saves on close
- Config location documented
- Assets path resolved correctly at runtime

Packaging (PyInstaller)

- Tested as standalone `.exe`
- Hidden imports in `.spec` file
- Assets bundled — path via `sys._MEIPASS`
- AV false positive considered — sign if possible
- Installer/uninstaller built

CLI Scripts

- `--help` flag
 - `--version` flag
 - Graceful `ctrl+c` — no ugly traceback
 - Meaningful exit codes (0 = success)
-

ANDROID

Touch Input

- Tap, double tap, long press — intentional
- Swipe gestures — no system gesture conflicts
- Pinch to zoom where expected
- Pull to refresh where appropriate
- Scroll inertia natural
- Touch targets minimum 48x48dp

Android-Specific

- Back button/gesture — every screen handled, no root crash
- Back stack correct
- Home — pause/resume correct
- Recent apps — `FLAG_SECURE` if sensitive
- Rotation — handled or locked intentionally
- Split-screen / multi-window considered
- Notifications permission (Android 13+)
- Runtime permissions in context — denial handled gracefully

UI Scaling

- Tested at 85%, 100%, 115%, 130% font scale

- No text clipping at large font scale
- Tested on 5" and 6.7" screens
- Keyboard avoidance — inputs above keyboard
- Safe area insets — nothing behind nav bar
- Status bar color matches theme
- Dark mode supported or explicitly out of scope

Stability

- No heavy work on main thread
- Low memory handled
- Lifecycle tested
- No full re-fetch on every resume
- APK/AAB size reasonable

Data

- Offline state — no crash, clear message
- Data survives process death
- No plaintext sensitive storage

APPLE (iOS / iPadOS)

Touch Input

- Tap, double tap, long press — intentional
- Left edge swipe (back) — not blocked
- In-app swipes — no system conflicts
- Pinch to zoom where expected
- Pull to refresh where appropriate
- Haptics — appropriate, not overused

iOS-Specific

- Safe area insets — nothing behind home indicator
- Notch / Dynamic Island — content clear
- Control Center swipe — no conflict
- Keyboard avoidance
- Numeric keyboard — Done button present
- Return key logical
- Rotation — handled or locked
- iPad — split view / slide over considered

UI Scaling

- Tested on SE (smallest) and Pro Max (largest)
- Dynamic Type — nothing clips at max size
- Status bar style set per screen
- Dark mode or explicitly out of scope

System

- No cleartext HTTP
- Info.plist privacy strings for all permissions
- Permissions in context — denial graceful
- Privacy manifest accurate

Performance

- Cold launch under 2 seconds
- Memory warnings handled
- Low power mode — nothing breaks

Data

- State restoration

- Offline handled
- iCloud — intentional yes or no

App Store

- Screenshots all device sizes
 - Description written
 - Keywords researched
 - Bundle ID, version, build number set
-
-



PHASE 7 — MODULE DOCUMENTATION

(Document Each Module As It's Built — Not After)

Module Breakdown Format

Every module gets documented in this format inside HANDOFF.md. This is the “fusion style” breakdown — every function accounted for.

```
---  
  
## MODULE: module_name.py  
**Responsibility:** One sentence — what this module does and nothing else  
**Depends on:** list of other modules it imports from  
**Used by:** list of modules that import from this one  
  
### Functions:  
  
#### function_name(param1, param2)  
- **What it does:** Plain English description  
- **Inputs:** param1 (type) — what it represents | param2 (type) — what it repres  
- **Returns:** what comes back and what type  
- **Touches:** what files, database tables, or other modules it reaches into  
- **Edge cases / quirks:** anything non-obvious  
- **Last modified:** date and what changed  
  
#### next_function_name(param)
```


...

Decision Log Format

Document every non-obvious architectural choice:

```
## DECISION LOG

### [Date] - Decision title
- **What was decided:**
- **Why:**
- **What was considered and rejected:**
- **Consequences / tradeoffs:**
```

Tech Debt Tracker Format

```
## TECH DEBT

| ID | Description | Why It Exists | Priority | Module |
|----|-----|-----|-----|-----|
| TD-001 | | | High/Med/Low | |
```

PHASE 8 — TESTING

(Before Ship — Every Time)

Testing Strategy

- Unit tests for all core business logic
- Manual test script written — exact steps, not just “test it”
- Regression test for every bug fixed — test written so it never comes back

- Tested on a clean machine / fresh install — not just dev environment
- All core user flows tested end to end manually
- Performance baseline verified:
 - Target load / startup time: _____
 - Known heavy operations identified and profiled
 - Pagination or lazy loading in place for large data sets
- Edge cases documented and tested:
 - Empty inputs
 - Maximum length inputs
 - Special characters
 - Network failure mid-operation
 - Low disk space / memory
- Beta / test user feedback loop if applicable

Self-Code Review (Before AI-Assisted Edits)

- Still matches original design mockup
 - All modules still doing only their one job
 - Nothing hardcoded that shouldn't be
 - Variable and function names still clear as project grew
 - No debug `print()` statements left in code
 - No TODO comments without a matching tech debt entry
-
-

PHASE 9 — RELEASE & DEPLOYMENT

(How It Ships — Written Before You Need It)

- Release build process documented step by step
 - Version bump process defined — where to update, in what order
 - Build tested on clean machine before every release
 - Changelog updated with all changes since last release
 - All known bugs documented — fix or document, never ship silent unknowns
 - Rollback plan defined — what happens if something breaks after ship
 - Distribution method confirmed:
 - Web: deployment target, CI/CD pipeline
 - Desktop: installer location, update mechanism
 - Mobile: store submission process
 - Auto-update or manual update check in place
 - Post-ship monitoring plan — how will you know if something broke
-
-

HANDOFF.md TEMPLATE

(Create This at Project Start — AI Updates Every Session)

Copy this to `HANDOFF.md` in the project root and fill it out. The AI must update this at the end of every session before closing.

```
# HANDOFF — [Project Name]
**Last updated:** [Date]
**Updated by:** [AI name and session summary]
**Project type:** [Web / Desktop / Mobile / Script]

---

## 🟦 CURRENT STATE

**Phase:** [0-Identity / 1-Setup / 2-Architecture / 3-Design / 4-Build / 5-Testin
**Current focus:** [What module or feature is actively being worked on]
```

 COMPLETED THIS SESSION

-
-
-

 SOON NEXT SESSION — DO THIS FIRST

- 1.
- 2.
- 3.

 BLOCKED / PENDING DECISIONS

- [] [Issue] — [What needs to be decided or resolved]

 KNOWN BUGS / TECH DEBT

ID	Description	Priority	Module	Notes
-----	-----	-----	-----	-----

 MASTER CHECKLIST STATUS

- [x] Phase 0 — Session Protocol
- [x] Phase 1 — Identity
- [] Phase 2 — Project Setup
- [] Phase 3 — Architecture
- [] Phase 4 — Design
- [] Phase 5 — Security
- [] Phase 6 — Build
- [] Phase 7 — Module Documentation
- [] Phase 8 — Testing
- [] Phase 9 — Release

 MODULE BREAKDOWN

[Fill in per module using format from Phase 7]

📖 DECISION LOG

[Date] - [Decision title]

- **Decided:**
- **Why:**
- **Rejected:**
- **Tradeoffs:**

⚙️ ENVIRONMENT & CONFIG REFERENCE

- **Ports used:**
- **Key env variables:**
- **Config file location:**
- **Build output location:**
- **Log file location:**

📦 DEPENDENCY MAP

[Which modules depend on which]

🔄 SESSION CHANGELOG (Last 5 Sessions)

Session [N] - [Date]

-

Session [N-1] - [Date]

-

PRE-SHIP FINAL GATE

(Every Item Must Be Checked Before Anything Ships)

- All phases complete

- HANDOFF.md current and accurate
- No console errors or warnings
- No hardcoded secrets or API keys anywhere in codebase
- `.env` is NOT committed to the repository
- Version number correct and visible in app
- README accurate and up to date
- CHANGELOG updated
- Dependencies pinned
- Logging working — correct levels, correct output
- Error handling verified — bad inputs, network failures, edge cases all handled
- Tested on clean machine / fresh install
- All core flows manually tested end to end
- All known bugs documented — none are silent unknowns
- Security review phase completed
- Performance baselines met
- Rollback plan documented

AI PROJECT MANAGER — Universal Edition Use this document to open every project session. Keep HANDOFF.md current. Build right the first time.